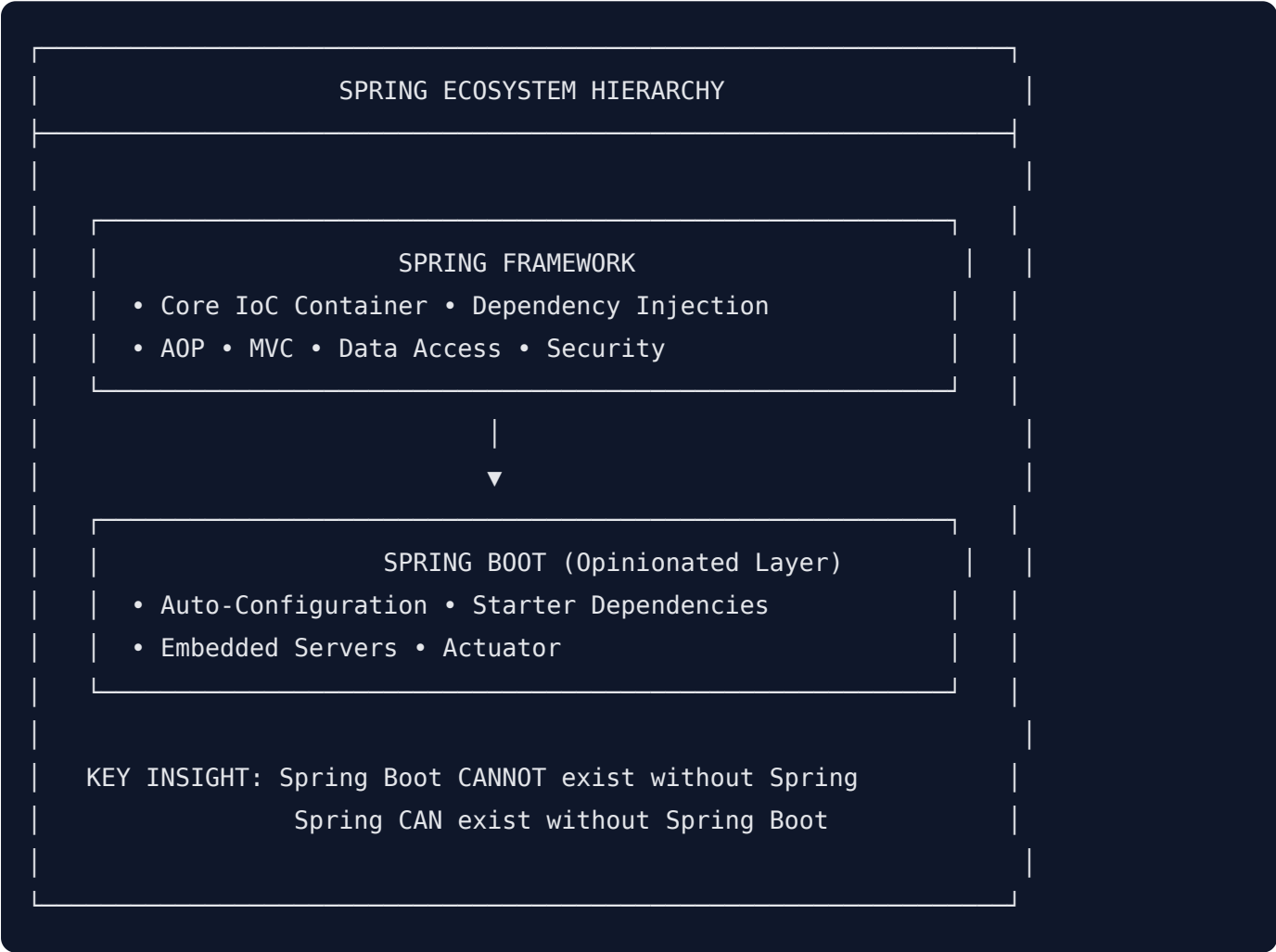


SPRING & SPRING BOOT DEEP DIVE — Advanced Interview Guide

For the 16-Year Experience Professional

PART 1: SPRING VS SPRING BOOT — THE BIG PICTURE

Core Relationship



Detailed Comparison Matrix

Dimension	Spring Framework	Spring Boot
Configuration	Manual XML/Java config	Auto-configuration + minimal properties

Dependency Management	Manual version resolution	Starter POMs with curated versions
Deployment	WAR to external server (Tomcat/JBoss)	Executable JAR with embedded server
Development Speed	Slower (configuration overhead)	Very fast (convention over configuration)
Control Level	High (everything explicit)	Moderate (opinionated defaults)
Use Case	Large enterprise, custom requirements	Microservices, cloud-native, rapid development
Learning Curve	Steeper	Gentler

When to Choose Which

FRAMEWORK SELECTION GUIDE

Choose SPRING FRAMEWORK when:

— You need fine-grained control over every configuration

— Legacy system integration requires specific setup

— Team is already experienced with traditional Spring

— Complex custom requirements not covered by Boot defaults

Choose SPRING BOOT when:

— Starting new microservices project

— Rapid development and prototyping

— Cloud-native or containerized deployment

— Reducing boilerplate configuration is priority

— Standard enterprise patterns suffice

PRO TIP: Most NEW projects (2024-2026) should start with Boot

PART 2: SPRING BOOT INTERNALS — HOW IT REALLY WORKS

Application Startup Flow

SPRING BOOT INTERNAL STARTUP SEQUENCE

@SpringBootApplication (Main Entry Point)



@EnableAutoConfiguration

- Reads META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
- Evaluates @Conditional annotations
- Registers auto-configuration classes



@ComponentScan

- Scans package for @Component, @Service, @Repository
- Uses index (if spring-context-indexer present)
- Registers found components in IoC container



@Configuration & @Bean Processing

- Parses @Configuration classes
- Executes @Bean methods (CGLIB proxied)
- Ensures singleton scope



IoC Container (ApplicationContext)

- Manages bean lifecycle (init, destroy)
- Handles dependency injection (@Autowired)
- BeanPostProcessors for AOP, etc.



Embedded Web Server (Tomcat/Jetty/Undertow) Startup

@Configuration vs @AutoConfiguration — Critical Distinction

Aspect	@Configuration	@AutoConfiguration
Introduced	Spring 3.0 (2009)	Spring Boot 2.7 (2022)
Primary Use	User-defined application beans	Library/Starter auto-configuration
Loading Order	After all auto-configurations	Before user configurations
Component Scanning	Participates	Opt-out by default
Condition Evaluation	Standard	Optimized + cached
Registration	Implicit via @ComponentScan	Via META-INF/spring/... imports file

Correct Usage Pattern

```
// ✅ CORRECT for user application configuration
@Configuration
@EnableConfigurationProperties(MyAppProperties.class)
public class MyAppConfig {
    @Bean
    public MyService myService() {
        return new MyService();
    }
}

// ✅ CORRECT for library/starter auto-configuration (Boot 2.7+)
@Configuration
@ConditionalOnClass(SomeLibrary.class)
@EnableConfigurationProperties(MyStarterProperties.class)
public class MyAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean // Let users override
    public SomeService someService() {
        return new SomeService();
    }
}
```

Auto-Configuration Loading Order Control

```
// Control execution order between auto-configurations
@Configuration(after = DataSourceAutoConfiguration.class)
@Configuration(before = WebMvcAutoConfiguration.class)
public class MyCustomAutoConfiguration {
    // Ensures this runs AFTER DataSource is set up
    // but BEFORE Web MVC configuration
}
```

PART 3: STARTER DEPENDENCIES DEEP DIVE

Common Starters and What They Include

Starter	Transitive Dependencies	Use Case
---------	-------------------------	----------

<code>spring-boot-starter-web</code>	Spring MVC, Jackson, Tomcat, Validation	REST APIs, web apps
<code>spring-boot-starter-webflux</code>	Spring WebFlux, Reactor Netty	Reactive applications
<code>spring-boot-starter-data-jpa</code>	Hibernate, Spring Data JPA, HikariCP	Database access with ORM
<code>spring-boot-starter-data-mongodb</code>	MongoDB driver, Spring Data MongoDB	Document database
<code>spring-boot-starter-security</code>	Spring Security Core, OAuth2 client	Authentication/Authorization
<code>spring-boot-starter-actuator</code>	Micrometer, health endpoints	Monitoring, metrics
<code>spring-boot-starter-cloud</code>	Spring Cloud context, discovery client	Microservices
<code>spring-boot-starter-test</code>	JUnit 5, Mockito, AssertJ, Testcontainers	Unit/integration testing

Custom Starter — When and How

WHEN TO CREATE CUSTOM STARTER

✓ Good candidates:

- Internal company libraries used across microservices
- Common security configuration (JWT validation)
- Standard logging/MDC setup
- Database/queue connection factories
- Shared DTOs and validation logic

✗ Not recommended:

- Single-use configuration
- Highly volatile business logic
- Domain-specific aggregates

PART 4: PRODUCTION-GRADE CONFIGURATION

Essential application.yml for Production

```
# =====
# PRODUCTION SPRING BOOT CONFIGURATION (APPLICATION.YAML)
# =====

server:
  port: 8080
  tomcat:
    max-connections: 10000      # Handle concurrent connections
    threads:
      max: 800                  # Worker threads (CPU cores * 2-4)
      min-spare: 100            # Ready for traffic spikes
    accept-count: 100           # Queue when threads busy
    connection-timeout: 20000   # 20 seconds max wait

spring:
  datasource:
    hikari:
      maximum-pool-size: 50      # NOT infinite—database has limits
      minimum-idle: 10
      connection-timeout: 30000
      idle-timeout: 600000
      max-lifetime: 1800000
      leak-detection-threshold: 60000 # Detect connection leaks

  jackson:
    time-zone: UTC               # No server-local time surprises
    date-format: yyyy-MM-dd HH:mm:ss
    serialization:
      write-dates-as-timestamps: false

  servlet:
    multipart:
      max-file-size: 100MB
      max-request-size: 100MB

  task:
    execution:
      pool:
        core-size: 8             # @Async thread pool
        max-size: 16
        queue-capacity: 100
        thread-name-prefix: async-
```

```
logging:
  level:
    com.mycompany: INFO
    org.springframework.web: WARN
  logback:
    rollingpolicy:
      max-file-size: 100MB
      max-history: 30
      total-size-cap: 3GB

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: when-authorized
  metrics:
    export:
      prometheus:
        enabled: true
```

PART 5: PERFORMANCE TUNING — PRODUCTION READY

Dependency Management for Fast Startup

STARTUP TIME OPTIMIZATION STRATEGIES

1. Remove Unused Dependencies

```
$ mvn dependency:analyze
```

Identifies declared but unused dependencies

2. Exclude Unnecessary Auto-Configurations

```
@SpringBootApplication(exclude = {  
    DataSourceAutoConfiguration.class,  
    SecurityAutoConfiguration.class  
})
```

3. Enable Lazy Initialization

```
spring.main.lazy-initialization=true
```

⚠ Warning: First request pays initialization cost

4. Use Spring Context Indexer

```
<dependency>  
  <groupId>org.springframework</groupId>  
  <artifactId>spring-context-indexer</artifactId>  
  <optional>true</optional>  
</dependency>
```

Generates META-INF/spring.components → eliminates scanning

JVM Tuning for Spring Boot

Heap Configuration

```
# CONTAINERIZED ENVIRONMENT (Docker/K8s with 1GB limit)
java -Xms600m -Xmx600m \
    -XX:+UseG1GC \
    -XX:MaxGCPauseMillis=200 \
    -jar app.jar

# DEDICATED SERVER (16GB available)
java -Xms8g -Xmx8g \
    -XX:+UseZGC \
    -XX:+HeapDumpOnOutOfMemoryError \
    -XX:HeapDumpPath=/var/log/app \
    -jar app.jar
```

Garbage Collector Selection Guide

GC	When to Use	Heap Size	Trade-off
G1GC (Default)	General purpose, predictable pauses	4-32GB	Balanced throughput/latency
ZGC	Ultra-low latency (<10ms pauses)	Any, excels at large	Slightly lower throughput
Parallel GC	Batch processing, throughput priority	Any	Higher throughput, longer pauses

Embedded Server Tuning

Tomcat Optimization

```
server:
  tomcat:
    threads:
      max: 200                # Default, adjust based on CPU cores
      min-spare: 10           # Keep ready for spikes
    max-connections: 10000    # Connections before queuing
    accept-count: 100         # Queue size when all threads busy
```

Alternative Embedded Servers

Server	Best For	Key Feature
Tomcat (default)	General purpose	Mature, widely understood

Jetty	Cloud-native, memory-constrained	Lower memory footprint
Undertow	High throughput	Non-blocking I/O, WebSocket support

Interview Insight: *"For most Spring Boot applications, default Tomcat works fine. Switch to Jetty if you're memory-constrained (<512MB heap). Undertow shines for WebSocket-heavy or extreme throughput scenarios."*

PART 6: SPRING BOOT ACTUATOR — PRODUCTION READY

Critical Actuator Endpoints

ACTUATOR ENDPOINTS MATRIX	
Endpoint	Purpose
/health	Liveness + Readiness probes for K8s
/info	Build info, git commit, custom metadata
/metrics	JVM memory, GC, thread, HTTP request metrics
/prometheus	Metrics in Prometheus format
/threaddump	Thread dump for deadlock analysis
/heapdump	Heap dump for memory leak investigation
/env	Environment properties (sensitive—secure it)
/configprops	All @ConfigurationProperties beans
/beans	All Spring beans in context
/mappings	Request mapping routes

Production Security Configuration

```

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus # NOT /env or /heapdump
        base-path: /internal/actuator           # Obscure from attackers
  endpoint:
    health:
      show-details: when-authorized
      probes:
        enabled: true # For Kubernetes

```

Kubernetes Probes Integration

```

# Deployment.yaml
livenessProbe:
  httpGet:
    path: /internal/actuator/health/liveness
    port: 8080
  initialDelaySeconds: 60
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /internal/actuator/health/readiness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 5

```

PART 7: SPRING BOOT WITH MICROSERVICES — SPRING CLOUD

Spring Cloud Component Map

SPRING CLOUD MICROSERVICES STACK

Spring Cloud Gateway (API Gateway)

- Routing • Rate limiting • Security filtering



Spring Cloud Netflix / Kubernetes Native

- Service Discovery (Eureka / K8s API)
- Client-side Load Balancing (Ribbon / Spring Cloud LoadBalancer)



Spring Cloud Circuit Breaker (Resilience4j)

- Retry • Timeout • Circuit Breaker • Rate Limiter



Spring Cloud Stream / Spring for Apache Kafka

- Event-driven communication
- Binders for Kafka, RabbitMQ



Spring Cloud Sleuth / Micrometer Tracing

- Distributed tracing (OpenTelemetry)
- W3C Trace Context propagation

Resilience4j Configuration (Circuit Breaker)

```
# application.yml
resilience4j:
  circuitbreaker:
    instances:
      payment-service:
        sliding-window-size: 10
        failure-rate-threshold: 50
        wait-duration-in-open-state: 30s
        permitted-number-of-calls-in-half-open-state: 5
        automatic-transition-from-open-to-half-open-enabled: true

  retry:
    instances:
      payment-service:
        max-attempts: 3
        wait-duration: 1s
        retry-exceptions:
          - org.springframework.dao.TransientDataAccessException
```

```
@CircuitBreaker(name = "payment-service", fallbackMethod = "fallbackPayment")
@Retry(name = "payment-service")
public PaymentResponse processPayment(PaymentRequest request) {
    return paymentClient.process(request);
}

public PaymentResponse fallbackPayment(PaymentRequest request, Exception e) {
    return PaymentResponse.pending("Payment queued for retry");
}
```

PART 8: TESTING DEEP DIVE

Testing Annotations Reference

Annotation	Use Case	Loads
@SpringBootTest	Full integration test	Full application context
@WebMvcTest	Controller layer only	Only web layer, @Controller, @ControllerAdvice
@DataJpaTest	Repository layer	JPA repositories, in-memory DB

@JsonTest	JSON serialization	Jackson/ObjectMapper
@DataRedisTest	Redis operations	Redis repositories
@RestClientTest	REST client testing	RestTemplate, WebClient

Mock vs Spy — Staff Level Understanding

```
// MOCK: Complete fake implementation (default: return null/empty)
@Mock
userRepository repository; // ALL methods return null/empty collections

when(repository.findById(1L)).thenReturn(Optional.of(user));

// SPY: Real object with ability to stub specific methods
@Spy
userService service; // REAL methods execute unless stubbed

doReturn("forced response").when(service).getExternalData(any());
// Other methods execute REAL implementation
```

PART 9: SPRING BOOT 3.x — WHAT'S DIFFERENT

Breaking Changes (Boot 2 → Boot 3)

Area	Boot 2	Boot 3
Java Baseline	Java 8/11	Java 17 minimum
Jakarta EE	javax.*	jakarta.*
Hibernate	5.x	6.x
Security	Spring Security 5	Spring Security 6 (lambda DSL)
Tracing	Spring Cloud Sleuth	Micrometer Tracing
GraalVM	Experimental	Native support

Jakarta EE Migration

```
// Boot 2 (Java EE)
import javax.persistence.Entity;
import javax.validation.constraints.NotNull;

// Boot 3 (Jakarta EE)
import jakarta.persistence.Entity;
import jakarta.validation.constraints.NotNull;
```

PART 10: INTERVIEW POWER PHRASES — STAFF LEVEL

Spring vs Spring Boot

"Spring Boot is NOT a replacement for Spring—it's an opinionated layer on top that provides auto-configuration, starter dependencies, and embedded servers. Spring Boot's biggest innovation is shifting from configuration files to conditional auto-configuration based on classpath."

Auto-Configuration Internals

"At startup, @EnableAutoConfiguration reads all META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports files, then evaluates @Conditional annotations. The optimization here is that conditions are evaluated in phases—class-level first, then method-level—with caching through spring-autoconfigure-metadata.json."

Lazy Initialization Trade-offs

"Setting spring.main.lazy-initialization=true reduces startup time by delaying bean creation until first use. But the first request becomes slower, and runtime failures may appear only after deployment. I prefer selective @Lazy for specific beans rather than blanket lazy initialization."

Connection Pool Sizing

*"The formula for HikariCP pool size is (core_count * 2) + spinning_disk_count. For most applications, 50 is too high—you'll get connection contention. Start with 10-20, monitor, adjust. The database has limits too."*

Embedded Server Selection

"Tomcat is fine for 95% of cases. Switch to Jetty if running memory-constrained (<512MB heap because of cloud costs). Undertow gives the best WebSocket performance. The choice reveals more about your operational constraints than technical preference."

Actuator in Production

"Actuator is not just for humans— /health/liveness and /health/readiness are essential for Kubernetes. We also use /metrics to feed Prometheus, and /heapdump on-demand for memory leak investigations. Never expose /env or /configprops without strict network restrictions."

Custom Starters

"We use custom starters for cross-cutting concerns—JWT validation, MDC logging setup, and standard HTTP client configuration. A proper starter uses @AutoConfiguration (not @Configuration) and exports conditionally with @ConditionalOnMissingBean to allow per-service overrides."

QUICK REFERENCE CARD

Essential Annotations

Annotation	Purpose
@SpringBootApplication	Main entry (composes @EnableAutoConfig + @ComponentScan + @Configuration)
@EnableAutoConfiguration	Enables auto-config mechanism
@ConfigurationProperties	Bind external config to typed objects
@ConditionalOnXxx	Conditional auto-config (Class, Bean, Property, MissingBean)
@RestControllerAdvice	Global exception handling
@Async + @EnableAsync	Asynchronous method execution

Production Checklist

- ❑ Actuator endpoints secured (not exposed publicly)
- ❑ Database connection pool sized appropriately
- ❑ Jackson timezone set to UTC
- ❑ Log rolling policy configured
- ❑ Heap size set (-Xms = -Xmx)
- ❑ Unused dependencies removed
- ❑ Unnecessary auto-configurations excluded
- ❑ Health probes for Kubernetes configured
- ❑ Distributed tracing enabled (if microservices)

Performance Baseline Numbers

Metric	Healthy Range
Startup time (simple service)	2-5 seconds
Startup time (complex service)	15-30 seconds
Memory footprint (minimum)	200-400 MB
Heap usage (steady state)	40-70% of -Xmx
GC pause time	< 200ms (G1GC) / < 10ms (ZGC)

This document covers Spring/Spring Boot at the depth expected for a 16-year experienced professional. Focus on understanding the **trade-offs** and **why** decisions are made, not just the syntax.

SPRING ANNOTATIONS MASTER REFERENCE

Complete Guide with Usage Patterns — For 16-Year Experience Professional

PART 1: STEREOTYPE ANNOTATIONS (Component Model)

Core Stereotypes

Annotation	Purpose	Meta-Annotation	Typical Layer
------------	---------	-----------------	---------------

@Component	Generic Spring-managed bean	None	Utility classes, wiring
@Service	Business logic marker	@Component	Service layer
@Repository	DAO marker with exception translation	@Component	Data access layer
@Controller	Web controller (returns view)	@Component	Web layer
@RestController	REST controller (returns data)	@Controller + @ResponseBody	REST API layer

Usage Patterns

```

// =====
// @Component - Generic Spring Bean
// =====
@Component
public class EmailValidator {
    private static final Pattern EMAIL_PATTERN =
        Pattern.compile("[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,6}$",
Pattern.CASE_INSENSITIVE);

    public boolean isValid(String email) {
        return EMAIL_PATTERN.matcher(email).matches();
    }
}

// =====
// @Service - Business Logic (Explicit intent)
// =====
@Service
@Slf4j
@Transactional(readOnly = true)
public class OrderService {

    private final OrderRepository orderRepository;
    private final PaymentClient paymentClient;

    public OrderService(OrderRepository orderRepository, PaymentClient
paymentClient) {
        this.orderRepository = orderRepository;
        this.paymentClient = paymentClient;
    }

    @Transactional
    public Order createOrder(OrderRequest request) {
        log.info("Creating order for customer: {}", request.getCustomerId());
        Order order = Order.fromRequest(request);
        return orderRepository.save(order);
    }
}

// =====
// @Repository - Data Access with Exception Translation
// =====

```

```

@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {
    // Spring automatically translates SQLException to DataAccessException
    List<Order> findByCustomerIdAndStatus(Long customerId, OrderStatus status);
}

// =====
// @RestController - REST API Controller
// =====
@RestController
@RequestMapping("/api/v1/orders")
@Slf4j
public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public OrderResponse createOrder(@Valid @RequestBody OrderRequest request) {
        return orderService.createOrder(request);
    }
}

```

PART 2: DEPENDENCY INJECTION ANNOTATIONS

Injection Annotations Matrix

Annotation	Resolution	Best For	Null Handling
@Autowired	By type, then by name	Constructor injection (recommended)	required=true by default
@Inject (JSR-330)	Same as @Autowired	Portable code (JSR-330 compatibility)	Optional with @Nullable

<code>@Resource</code> (JSR-250)	By name first, then by type	Java EE legacy, named resources	Required by default
<code>@Qualifier</code>	Disambiguates multiple beans	When multiple beans of same type	N/A
<code>@Primary</code>	Preferred bean when ambiguous	Default implementation override	N/A

Constructor Injection (Recommended Pattern)

```

// =====
// RECOMMENDED: Constructor Injection (Immutable, testable)
// =====
@RestController
public class OrderController {

    private final OrderService orderService;
    private final PaymentService paymentService;
    private final NotificationClient notificationClient;

    // @Autowired optional on single constructor (Spring 4.3+)
    public OrderController(
        OrderService orderService,
        PaymentService paymentService,
        NotificationClient notificationClient) {
        this.orderService = orderService;
        this.paymentService = paymentService;
        this.notificationClient = notificationClient;
    }
}

// =====
// Field Injection (NOT recommended - hard to test)
// =====
@RestController
public class OrderController {

    @Autowired // ❌ Reflection-based, not testable easily
    private OrderService orderService;
}

// =====
// Setter Injection (Optional dependencies)
// =====
@RestController
public class OrderController {

    private AuditService auditService; // optional dependency

    @Autowired(required = false)
    public void setAuditService(AuditService auditService) {
        this.auditService = auditService;
    }
}

```

```

    }
}

// =====
// @Qualifier - Multiple beans of same type
// =====
@Service
public class PaymentService {

    @Autowired
    @Qualifier("primaryPaymentGateway")
    private PaymentGateway paymentGateway;

    @Autowired
    @Qualifier("fallbackPaymentGateway")
    private PaymentGateway fallbackGateway;
}

// =====
// @Primary - Define default implementation
// =====
@Component
@Primary
public class PrimaryPaymentGateway implements PaymentGateway {
    // This bean is chosen when no @Qualifier specified
}

@Component
public class SecondaryPaymentGateway implements PaymentGateway {
    // Explicit @Qualifier needed to use this
}

```

PART 3: CONFIGURATION ANNOTATIONS

Configuration Annotations

Annotation	Purpose	Key Feature
@Configuration	Define bean definitions	Full configuration class

@Bean	Declare Spring bean inside @Configuration	Method returns bean instance
@ConfigurationProperties	Bind external properties to typed object	Type-safe configuration
@PropertySource	Load properties file	External configuration source
@Profile	Conditional bean activation per environment	Environment-specific beans
@Conditional	Custom conditional bean registration	Advanced condition logic
@Import	Import additional configuration classes	Modular configuration

Configuration Patterns

```
// =====
// @Configuration + @Bean - Full Configuration Class
// =====
@Configuration
@EnableConfigurationProperties(AppProperties.class)
@Slf4j
public class AppConfig {

    @Bean
    @Profile("development")
    public DataSource devDataSource() {
        log.info("Creating development H2 database");
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2)
            .addScript("schema.sql")
            .build();
    }

    @Bean
    @Profile("production")
    @ConditionalOnProperty(name = "database.type", havingValue = "postgres")
    public DataSource prodDataSource(AppProperties properties) {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl(properties.getDatabase().getUrl());
        config.setUsername(properties.getDatabase().getUsername());
        config.setPassword(properties.getDatabase().getPassword());
        config.setMaximumPoolSize(properties.getDatabase().getPoolSize());
        return new HikariDataSource(config);
    }

    @Bean
    @DependsOn({"dataSource", "cacheManager"}) // Ensure order
    public InitializationService initializationService() {
        return new InitializationService();
    }
}

// =====
// @ConfigurationProperties - Type-safe Configuration
// =====
@ConfigurationProperties(prefix = "app.payment")
@Component
```

```

@Data
public class PaymentProperties {

    private Gateway gateway = new Gateway();
    private Retry retry = new Retry();
    private CircuitBreaker circuitBreaker = new CircuitBreaker();

    @Data
    public static class Gateway {
        private String url = "https://default.payment.com";
        private int timeoutSeconds = 30;
        private String apiKey;
    }

    @Data
    public static class Retry {
        private int maxAttempts = 3;
        private Duration initialDelay = Duration.ofSeconds(1);
        private double multiplier = 2.0;
    }

    @Data
    public static class CircuitBreaker {
        private int failureThreshold = 5;
        private Duration timeout = Duration.ofSeconds(30);
    }
}

// application.yml
// app:
//   payment:
//     gateway:
//       url: https://prod.payment.com
//       timeout-seconds: 10
//     retry:
//       max-attempts: 5
//       initial-delay: 500ms

// =====
// @PropertySource - Custom Properties File
// =====
@Configuration
@PropertySource(value = "classpath:secrets.properties", ignoreResourceNotFound =

```

```

true)
@PropertySource(value = "file:/etc/app/secrets.properties", ignoreResourceNotFound
= true)
public class SecretsConfig {

    @Value("${encryption.key}")
    private String encryptionKey;

    @Value("${jwt.secret:defaultSecret}") // with default value
    private String jwtSecret;
}

// =====
// @Import - Modular Configuration
// =====
@Configuration
@Import({DatabaseConfig.class, CacheConfig.class, SecurityConfig.class})
public class RootConfig {
    // Aggregates multiple configuration classes
}

```

Conditional Annotations (Spring Boot)

```
// =====
// CONDITIONAL ANNOTATIONS - Production Patterns
// =====

@Configuration
public class ConditionalConfig {

    // Conditional on class presence
    @Bean
    @ConditionalOnClass(name = "org.apache.kafka.clients.producer.KafkaProducer")
    public KafkaTemplate<String, Object> kafkaTemplate() {
        return new KafkaTemplate<>();
    }

    // Conditional on missing bean
    @Bean
    @ConditionalOnMissingBean(ObjectMapper.class)
    public ObjectMapper defaultObjectMapper() {
        return new ObjectMapper()
            .registerModule(new JavaTimeModule())
            .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
    }

    // Conditional on property value
    @Bean
    @ConditionalOnProperty(name = "cache.enabled", havingValue = "true",
matchIfMissing = false)
    public CacheManager cacheManager() {
        return new CaffeineCacheManager();
    }

    // Conditional on web application
    @Bean
    @ConditionalOnWebApplication
    public WebMvcConfigurer webMvcConfigurer() {
        return new CustomWebMvcConfigurer();
    }

    // Conditional on expression
    @Bean
    @ConditionalOnExpression("'${app.environment}' != 'test' &&
${app.monitoring.enabled:true}")

```

```

public MonitoringService monitoringService() {
    return new MonitoringService();
}

// Java 8+ Optional pattern
@Bean
@ConditionalOnJava(JavaVersion.SEVENTEEN)
public Java17FeatureService java17Service() {
    return new Java17FeatureService();
}

// Custom annotation combining conditions
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@ConditionalOnClass(RedisConnectionFactory.class)
@ConditionalOnProperty(name = "redis.enabled", havingValue = "true")
public @interface ConditionalOnRedisEnabled {
}

@Bean
@ConditionalOnRedisEnabled
public RedisTemplate<String, Object> redisTemplate() {
    return new RedisTemplate<>();
}
}

```

PART 4: WEB & REST ANNOTATIONS

Request Mapping Annotations

Annotation	Purpose	Default Method
@RequestMapping	General mapping	GET (if method not specified)
@GetMapping	HTTP GET	GET
@PostMapping	HTTP POST	POST
@PutMapping	HTTP PUT	PUT
@DeleteMapping	HTTP DELETE	DELETE

@PatchMapping	HTTP PATCH	PATCH
---------------	------------	-------

Parameter Binding Annotations

Annotation	Binds To	Example
@PathVariable	URL template variable	/orders/{id} → @PathVariable Long id
@RequestParam	Query parameter	?page=1&size=10 → @RequestParam int page
@RequestBody	HTTP request body	JSON body → @RequestBody Order order
@RequestHeader	HTTP header	Authorization: Bearer xxx → @RequestHeader String auth
@CookieValue	Cookie value	JSESSIONID=abc → @CookieValue String sessionId
@ModelAttribute	Form data or query params	Form POST → @ModelAttribute User user
@MatrixVariable	Matrix variable	/cars;color=red;year=2020

Complete REST Controller Patterns

```
// =====
// COMPLETE REST CONTROLLER – PRODUCTION PATTERN
// =====
@RestController
@RequestMapping("/api/v1/orders")
@Slf4j
@Validated // Enable method-level validation
public class OrderController {

    private final OrderService orderService;
    private final OrderMapper orderMapper;

    public OrderController(OrderService orderService, OrderMapper orderMapper) {
        this.orderService = orderService;
        this.orderMapper = orderMapper;
    }

    // =====
    // GET with Path Variable + Request Params
    // =====
    @GetMapping("/{orderId}")
    public ResponseEntity<OrderResponse> getOrder(
        @PathVariable("orderId") Long id,
        @RequestParam(value = "includeDetails", defaultValue = "false")
boolean includeDetails,
        @RequestHeader(value = "X-Request-ID", required = false) String
requestId) {

        log.info("Fetching order {} (request-id: {})", id, requestId);
        Order order = orderService.findById(id);
        OrderResponse response = orderMapper.toResponse(order, includeDetails);
        return ResponseEntity.ok(response);
    }

    // =====
    // GET with Pagination
    // =====
    @GetMapping
    public Page<OrderSummary> listOrders(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size,
        @RequestParam(required = false) OrderStatus status,
```

```

        @RequestParam(defaultValue = "createdAt") String sortBy,
        @PageableDefault(size = 20, sort = "createdAt", direction =
Sort.Direction.DESC) Pageable pageable) {

    return orderService.findAll(status, pageable);
}

// =====
// POST with Validation
// =====
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public OrderResponse createOrder(
    @Valid @RequestBody OrderRequest request,
    @RequestHeader("Authorization") String authToken) {

    // @Valid triggers validation automatically
    Order order = orderService.createOrder(request, authToken);
    return orderMapper.toResponse(order, false);
}

// =====
// PUT (Full Update)
// =====
@PutMapping("/{orderId}")
public OrderResponse updateOrder(
    @PathVariable Long orderId,
    @Valid @RequestBody OrderUpdateRequest request) {

    Order updated = orderService.updateOrder(orderId, request);
    return orderMapper.toResponse(updated, false);
}

// =====
// PATCH (Partial Update)
// =====
@PatchMapping("/{orderId}/status")
public OrderResponse updateOrderStatus(
    @PathVariable Long orderId,
    @RequestBody StatusUpdateRequest request) {

    Order updated = orderService.updateStatus(orderId, request.getStatus());
    return orderMapper.toResponse(updated, false);
}

```

```

    }

    // =====
    // DELETE
    // =====
    @DeleteMapping("/{orderId}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteOrder(@PathVariable Long orderId) {
        orderService.deleteOrder(orderId);
    }
}

// =====
// @ModelAttribute - Form Binding (Traditional MVC)
// =====
@Controller
@RequestMapping("/users")
public class UserController {

    @ModelAttribute("countries")
    public List<String> populateCountries() {
        // This attribute available to ALL handlers in this controller
        return List.of("USA", "Canada", "Mexico");
    }

    @PostMapping("/register")
    public String registerUser(@ModelAttribute UserRegistrationForm form, Model
model) {
        // form populated from form parameters
        userService.register(form);
        model.addAttribute("message", "Registration successful");
        return "success";
    }
}

```

PART 5: VALIDATION ANNOTATIONS (Bean Validation)

Jakarta Validation Annotations (javax.validation → jakarta.validation)

Annotation	Purpose	Example
<code>@NotNull</code>	Value cannot be null	<code>@NotNull String name</code>
<code>@NotEmpty</code>	CharSequence/Collection/Map not null and not empty	<code>@NotEmpty List<Item> items</code>
<code>@NotBlank</code>	String not null and trimmed length > 0	<code>@NotBlank String username</code>
<code>@Size</code>	Size between min and max	<code>@Size(min=2, max=50) String name</code>
<code>@Min</code> / <code>@Max</code>	Numeric minimum/maximum	<code>@Min(0) @Max(100) int percent</code>
<code>@Pattern</code>	Regex pattern match	<code>@Pattern(regex="^[A-Z]+\$") String code</code>
<code>@Email</code>	Valid email format	<code>@Email String email</code>
<code>@Past</code> / <code>@Future</code>	Date in past/future	<code>@Past LocalDate birthDate</code>
<code>@Positive</code> / <code>@Negative</code>	Positive or negative number	<code>@Positive int quantity</code>
<code>@Digits</code>	Integer/fraction digit limit	<code>@Digits(integer=10, fraction=2) BigDecimal amount</code>

Custom Validation Example

```
// =====
// DTO with Validation Annotations
// =====
@Data
public class OrderRequest {

    @NotBlank(message = "Customer ID is required")
    private String customerId;

    @NotNull(message = "Order items cannot be null")
    @Size(min = 1, message = "Order must have at least one item")
    private List<OrderItem> items;

    @Positive(message = "Total amount must be positive")
    private BigDecimal totalAmount;

    @Email(message = "Invalid email format")
    @NotBlank(message = "Email is required")
    private String email;

    @Pattern(regex = "^[A-Z]{2}[0-9]{6}$", message = "Invalid promotion code
format")
    private String promotionCode;

    @Future(message = "Delivery date must be in the future")
    private LocalDate deliveryDate;
}

// =====
// Group Validation – Different rules for different operations
// =====
public class ValidationGroups {
    public interface Create {}
    public interface Update {}
    public interface Patch {}
}

@Data
public class ProductRequest {

    @NotNull(groups = {Create.class, Update.class})
    private Long id;
}
```

```

@NotBlank(groups = {Create.class, Update.class, Patch.class})
private String name;

@Positive(groups = {Create.class, Update.class})
private BigDecimal price;

@Null(groups = Create.class) // Can't set on create
@Positive(groups = Update.class)
private BigDecimal discountedPrice;
}

@RestController
public class ProductController {

    @PostMapping("/products")
    public Product create(@Validated(ValidationGroups.Create.class) @RequestBody
ProductRequest request) {
        return productService.create(request);
    }

    @PutMapping("/products/{id}")
    public Product update(@Validated(ValidationGroups.Update.class) @RequestBody
ProductRequest request) {
        return productService.update(request);
    }
}

// =====
// Custom Validation Annotation
// =====
@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = UniqueUsernameValidator.class)
@Documented
public @interface UniqueUsername {
    String message() default "Username already exists";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

@Component
public class UniqueUsernameValidator implements

```

```
ConstraintValidator<UniqueUsername, String> {

    @Autowired
    private UserRepository userRepository;

    @Override
    public boolean isValid(String username, ConstraintValidatorContext context) {
        if (username == null) return true;
        return !userRepository.existsByUsername(username);
    }
}

// Usage
@Data
public class UserRegistration {
    @UniqueUsername
    @NotBlank
    private String username;
}
```

PART 6: TRANSACTION MANAGEMENT ANNOTATIONS

@Transactional Deep Dive

```
// =====
// @TRANSACTIONAL – COMPLETE REFERENCE
// =====

@Service
public class OrderService {

    // =====
    // Basic Usage – Class Level
    // =====
    @Transactional(readOnly = true) // All read-only methods inherit
    public class ReadOnlyService {

        @Transactional // Overrides to read-write
        public Order createOrder(OrderRequest request) {
            // Transaction starts here
            Order order = orderRepository.save(new Order());
            paymentService.process(order); // Propagates transaction
            return order;
            // Commit on successful return
        }

        @Transactional(readOnly = true)
        public Order findById(Long id) {
            return orderRepository.findById(id).orElseThrow();
        }
    }

    // =====
    // Propagation Levels
    // =====
    @Transactional(propagation = Propagation.REQUIRED) // DEFAULT – Join or
create
    public void requiredExample() {}

    @Transactional(propagation = Propagation.REQUIRES_NEW) // Always suspend
current, create new
    public void requiresNewExample() {
        // Suspends existing transaction, starts new one
        auditService.log("operation"); // Logs even if outer transaction rolls
back
    }
}
```

```

    @Transactional(propagation = Propagation.NESTED) // Savepoint within existing
transaction
    public void nestedExample() throws Exception {
        // Creates savepoint, can roll back partial work
        try {
            riskyOperation();
        } catch (Exception e) {
            TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();
            // Only nested part rolls back
        }
    }

    @Transactional(propagation = Propagation.SUPPORTS) // Execute within
transaction if exists, else non-transactional
    public void supportsExample() {
        // Fine for read queries
    }

    @Transactional(propagation = Propagation.NOT_SUPPORTED) // Suspend
transaction, execute non-transactional
    public void notSupportedExample() {
        // Long-running non-transactional operation
        generateLargeReport();
    }

    @Transactional(propagation = Propagation.MANDATORY) // Must have existing
transaction
    public void mandatoryExample() {
        // Fails if called without transaction
    }

    @Transactional(propagation = Propagation.NEVER) // Must NOT have existing
transaction
    public void neverExample() {
        // Fails if called within transaction
    }

    // =====
    // Isolation Levels
    // =====

    // READ UNCOMMITTED – Dirty reads possible (lowest isolation)

```

```

@Transactional(isolation = Isolation.READ_UNCOMMITTED)
public void readUncommittedExample() {
    // Sees uncommitted changes from other transactions
    // ❌ Don't use – data consistency issues
}

// READ COMMITTED – Default in PostgreSQL, SQL Server
@Transactional(isolation = Isolation.READ_COMMITTED)
public void readCommittedExample() {
    // Only sees committed data
    // ✅ Good default for most OLTP systems
}

// REPEATABLE READ – Default in MySQL
@Transactional(isolation = Isolation.REPEATABLE_READ)
public void repeatableReadExample() {
    // Same query returns same data within transaction
    // Prevents non-repeatable reads
}

// SERIALIZABLE – Highest isolation, lowest concurrency
@Transactional(isolation = Isolation.SERIALIZABLE)
public void serializableExample() {
    // Complete isolation, no phantom reads
    // Use: Financial reconciliation, inventory allocation
}

// =====
// Rollback Rules
// =====

@Transactional(rollbackFor = {BusinessException.class,
DataAccessException.class})
public void specificRollbackExample() throws BusinessException {
    // Rolls back only on these exceptions
    if (invalidState) {
        throw new BusinessException("Invalid");
    }
    // RuntimeException triggers rollback by default
}

@Transactional(noRollbackFor = {OptimisticLockException.class})
public void noRollbackExample() {

```

```

        // OptimisticLockException won't cause rollback
        // Useful for retry scenarios
    }

    @Transactional(rollbackForClassName = {"BusinessException",
"ValidationException"})
    public void rollbackByClassNameExample() throws Exception {
        // Same as rollbackFor but with class names (for AOP proxies)
    }

    // =====
    // Timeout
    // =====

    @Transactional(timeout = 30) // Seconds
    public void timeoutExample() {
        // Transaction will be rolled back after 30 seconds
        // Prevents long-running transactions locking resources
    }

    // =====
    // Advanced – Transaction Template (Programmatic Control)
    // =====
    @Service
    public class AdvancedOrderService {

        private final TransactionTemplate transactionTemplate;

        public AdvancedOrderService(TransactionTemplate transactionTemplate) {
            this.transactionTemplate = transactionTemplate;
        }

        public Order createOrderWithRetry(OrderRequest request) {
            return transactionTemplate.execute(status -> {
                try {
                    Order order = createOrderInternal(request);
                    return order;
                } catch (OptimisticLockException e) {
                    status.setRollbackOnly();
                    throw new RetryableException(e);
                }
            });
        }
    }

```

```
}  
}
```

PART 7: CACHE ANNOTATIONS

Spring Cache Annotations

Annotation	Purpose	Example
<code>@EnableCaching</code>	Enable cache abstraction	<code>@SpringBootApplication @EnableCaching</code>
<code>@Cacheable</code>	Store result in cache	<code>@Cacheable("products")</code>
<code>@CacheEvict</code>	Remove from cache	<code>@CacheEvict(value="products", key="#id")</code>
<code>@CachePut</code>	Update cache without method execution	<code>@CachePut(value="products", key="#product.id")</code>
<code>@Caching</code>	Group multiple cache operations	Multiple evicts + caches
<code>@CacheConfig</code>	Class-level cache configuration	Shared cache names, key generator

Complete Caching Patterns

```
// =====
// CACHE CONFIGURATION
// =====
@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        CaffeineCacheManager cacheManager = new CaffeineCacheManager();
        cacheManager.setCaffeine(Caffeine.newBuilder()
            .expireAfterWrite(10, TimeUnit.MINUTES)
            .expireAfterAccess(5, TimeUnit.MINUTES)
            .maximumSize(1000)
            .recordStats());
        return cacheManager;
    }

    @Bean
    public KeyGenerator customKeyGenerator() {
        return (target, method, params) -> {
            return target.getClass().getSimpleName() + "_" +
                method.getName() + "_" +
                Arrays.deepHashCode(params);
        };
    }
}

// =====
// @Cacheable – Store and Retrieve
// =====
@Service
@CacheConfig(cacheNames = "products", keyGenerator = "customKeyGenerator")
public class ProductService {

    // Simple caching
    @Cacheable(value = "products", key = "#id")
    public Product findById(Long id) {
        simulateSlowDatabase();
        return productRepository.findById(id).orElseThrow();
    }
}
```

```

// Conditional caching
@Cacheable(value = "products", condition = "#id > 100", unless =
"#result.price > 1000")
public Product findWithCondition(Long id) {
    return productRepository.findById(id).orElseThrow();
    // Caches only if id > 100 AND result price ≤ 1000
}

// Cache with list of keys
@Cacheable(value = "products", key = "{#category, #status}")
public List<Product> findByCategoryAndStatus(String category, ProductStatus
status) {
    return productRepository.findByCategoryAndStatus(category, status);
}

// Sync – Prevents cache stampede
@Cacheable(value = "products", key = "#id", sync = true)
public Product findByIdSync(Long id) {
    // Multiple concurrent calls wait for one to populate cache
    return productRepository.findById(id).orElseThrow();
}
}

// =====
// @CachePut – Force Cache Update
// =====
@Service
public class ProductUpdateService {

    // Always executes method, always updates cache
    @CachePut(value = "products", key = "#product.id")
    public Product updateProduct(Product product) {
        return productRepository.save(product);
        // Cache updated with new value
    }
}

// =====
// @CacheEvict – Remove from Cache
// =====
@Service
public class ProductDeleteService {

```

```

// Remove single entry
@CacheEvict(value = "products", key = "#id")
public void deleteProduct(Long id) {
    productRepository.deleteById(id);
}

// Remove all entries from cache
@CacheEvict(value = "products", allEntries = true)
public void clearAllProducts() {
    // Useful after bulk operations
}

// Evict before method execution
@CacheEvict(value = "products", key = "#id", beforeInvocation = true)
public Product updateWithFreshCache(Long id, ProductUpdateRequest request) {
    // Removes old cache BEFORE method executes
    return productRepository.save(updatedProduct);
}
}

// =====
// @Caching – Multiple Operations
// =====
@Service
public class BulkCacheService {

    @Caching(
        put = {
            @CachePut(value = "products", key = "#result.id"),
            @CachePut(value = "productsByName", key = "#result.name")
        },
        evict = {
            @CacheEvict(value = "productList", allEntries = true)
        }
    )
    public Product createProduct(Product product) {
        // Updates multiple caches, evicts list cache
        return productRepository.save(product);
    }

    @Caching(evict = {
        @CacheEvict(value = "products", key = "#id"),
        @CacheEvict(value = "productsByName", allEntries = true)
    })

```

```
    })  
    public void deleteWithMultipleEvicts(Long id) {  
        productRepository.deleteById(id);  
    }  
}
```

PART 8: ASYNCHRONOUS & SCHEDULING ANNOTATIONS

@Async & @EnableAsync

```
// =====
// ASYNC CONFIGURATION
// =====
@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    @Bean(name = "taskExecutor")
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(25);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-");
        executor.setRejectedExecutionHandler(new
ThreadPoolExecutor.CallerRunsPolicy());
        executor.setWaitForTasksToCompleteOnShutdown(true);
        executor.setAwaitTerminationSeconds(60);
        executor.initialize();
        return executor;
    }

    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (ex, method, params) ->
            log.error("Async method {} failed with exception: {}",
method.getName(), ex.getMessage());
    }
}

// =====
// @Async USAGE PATTERNS
// =====
@Service
@Slf4j
public class NotificationService {

    // Fire and forget
    @Async
    public void sendEmail(String to, String subject, String body) {
        // Executes in separate thread
    }
}
```

```

        // Caller doesn't wait
        try {
            emailClient.send(to, subject, body);
            log.info("Email sent to {}", to);
        } catch (Exception e) {
            log.error("Email failed to {}", to, e);
        }
    }

    // Async with return value (Future)
    @Async
    public CompletableFuture<String> processFileAsync(String filePath) {
        String result = fileProcessor.process(filePath);
        return CompletableFuture.completedFuture(result);
    }

    // Async with specific executor
    @Async("mailExecutor")
    public void sendBulkEmails(List<String> recipients) {
        // Uses dedicated email executor
        recipients.forEach(this::sendIndividualEmail);
    }

    // ListenableFuture (Spring 4.3+)
    @Async
    public ListenableFuture<Report> generateReport(ReportRequest request) {
        Report report = reportGenerator.generate(request);
        return new AsyncResult<>(report);
    }
}

// =====
// CALLER PATTERN – Using Async Methods
// =====
@RestController
public class ReportController {

    private final NotificationService notificationService;

    public ReportController(NotificationService notificationService) {
        this.notificationService = notificationService;
    }
}

```

```
@PostMapping("/reports/generate")
public ResponseEntity<String> generateReport(@RequestBody ReportRequest
request) {
    // Fire async
    notificationService.sendEmail(request.getEmail(), "Report Ready", "Your
report is processing");

    // Wait for async result
    CompletableFuture<String> future =
notificationService.processFileAsync(request.getFilePath());
    String result = future.get(30, TimeUnit.SECONDS); // Block with timeout

    return ResponseEntity.ok(result);
}
}
```

@Scheduled & @EnableScheduling

```
// =====
// SCHEDULING CONFIGURATION
// =====
@Configuration
@EnableScheduling
public class SchedulingConfig {

    @Bean
    public TaskScheduler taskScheduler() {
        ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
        scheduler.setPoolSize(5);
        scheduler.setThreadNamePrefix("scheduled-");
        scheduler.setWaitForTasksToCompleteOnShutdown(true);
        scheduler.setAwaitTerminationSeconds(60);
        return scheduler;
    }
}

// =====
// @SCHEDULED USAGE PATTERNS
// =====
@Component
@Slf4j
public class ScheduledTasks {

    // Fixed delay – between completion of last execution and start of next
    @Scheduled(fixedDelay = 5000) // 5 seconds after previous completion
    public void fixedDelayTask() {
        log.info("Running with fixed delay");
        // Execution time doesn't affect schedule start
    }

    // Fixed rate – between start times (may overlap)
    @Scheduled(fixedRate = 60000) // Every 60 seconds regardless of duration
    public void fixedRateTask() {
        log.info("Running with fixed rate");
        // ⚠ Risk of overlapping if task takes > interval
    }

    // Initial delay before first execution
    @Scheduled(fixedRate = 60000, initialDelay = 30000) // Start after 30 seconds
    public void delayedStartTask() {

```

```
        log.info("First run after 30 seconds, then every minute");
    }

    // Cron expression (most flexible)
    @Scheduled(cron = "0 0 2 * * MON-FRI") // 2 AM every weekday
    public void weeklyReportTask() {
        // Generate nightly reports
        reportService.generateDailyReport();
    }

    // Cron with timezone
    @Scheduled(cron = "0 0 9 * * *", zone = "America/New_York")
    public void usMarketOpenTask() {
        // Runs at 9 AM US Eastern time
    }

    // Using ZoneDateTime for dynamic scheduling
    @Scheduled(cron = "0 #{new java.util.Date().getHours()} * * * ?")
    public void dynamicCronTask() {
        // Cron expression can use placeholders
        log.info("Dynamic cron task");
    }
}
```